

Projeto Café na Taverna

Equipe de Desenvolvimento: Matheus Castilhos, Mario da Fonseca, Ryan Kologeski.

1. Introdução

O projeto **Café na Taverna** consiste em um sistema web completo desenvolvido para centralizar, gerenciar e divulgar o conteúdo do podcast homônimo. Atualmente hospedado na Anchor.fm, o podcast terá seus dados integrados a uma plataforma própria, permitindo a reprodução direta de episódios, atualização automática via feed RSS e uma área administrativa para gestão de conteúdos complementares.

2. Visão Geral do Sistema

O sistema é composto por uma API robusta desenvolvida em Node.js com o framework Express, utilizando MongoDB como banco de dados principal. A arquitetura foca em escalabilidade e portabilidade, sendo provisionada via Docker e protegida por autenticação baseada em tokens JWT.

Tecnologias Utilizadas

- **Ambiente de Execução:** Node.js 18+
- **Framework Web:** Express
- **Persistência de Dados:** MongoDB com ODM Mongoose
- **Segurança:** JSON Web Tokens (JWT) para autenticação e autorização
- **Integração:** Manipuladores de RSS para sincronização de episódios
- **Containerização:** Docker e Docker Compose

3. Arquitetura da Aplicação

A aplicação segue um padrão modular baseado em camadas de responsabilidade:

3.1. Estrutura de Diretórios

- **src/server.js:** Ponto de entrada da aplicação, onde são configurados os middlewares globais e a inicialização do servidor.
- **src/config/database.js:** Gerenciamento da conexão com o banco de dados MongoDB via variáveis de ambiente.
- **src/middleware/authMiddleware.js:** Filtro de segurança para validação de tokens e controle de privilégios de administrador.

- **src/models/**: Definição dos esquemas de dados (Schemas).
- **src/routes/**: Definição dos endpoints da API organizados por domínio.

3.2. Modelagem de Dados (Schemas)

- **User (Usuário)**: Armazena nome, e-mail, senha criptografada e uma flag booleana de administrador.
- **Episode (Episódio)**: Armazena título, descrição, link de áudio, data de publicação, thumbnail, duração e metadados adicionais.

4. Segurança e Autenticação

O acesso a rotas sensíveis (CRUD de episódios e gerenciamento de usuários) é restrito a administradores.

- **Mecanismo**: Autenticação via `Authorization: Bearer <token>`.
- **Validação**: O `authMiddleware` intercepta as requisições, verifica a validade do JWT e as permissões do usuário antes de liberar o acesso ao controlador.
- **Bootstrap**: A API permite a criação automática de um administrador inicial através de variáveis de ambiente pré-configuradas no arquivo `.env`.

5. Especificação de Endpoints

5.1. Autenticação (`/auth`)

- `POST /auth/register`: Registro de novos usuários.
- `POST /auth/login`: Autenticação e geração de token JWT.

5.2. Episódios (`/episodes`)

- `GET /episodes`: Lista todos os episódios (Acesso público).
- `GET /episodes/:id`: Detalha um episódio específico (Acesso público).
- `POST /episodes`: Cadastro manual de episódios (Acesso: Admin).
- `PUT /episodes/:id`: Edição de metadados de episódios (Acesso: Admin).
- `DELETE /episodes/:id`: Remoção de episódios (Acesso: Admin).

5.3. Sincronização RSS (`/rss`)

- `GET /rss/fetch`: Endpoint responsável por consumir o feed externo da Anchor.fm e popular automaticamente a coleção de episódios no MongoDB.

5.4. Usuários e Sistema (`/users` e `/get_api`)

- **GET /users**: Lista usuários cadastrados (Acesso: Admin).
- **PUT /users/:id**: Edição de permissões e dados de usuários (Acesso: Admin).
- **GET /get_api**: Retorna informações públicas sobre o status e metadados da API.

6. Fluxo de Funcionamento e Deploy

O fluxo operacional inicia com a configuração do ambiente via Docker. Uma vez que o serviço está ativo:

1. O Administrador realiza o login e utiliza o endpoint **/rss/fetch** para sincronizar os dados do podcast original.
2. Os dados são parseados e armazenados no MongoDB.
3. A plataforma web consome os endpoints públicos para exibir o conteúdo aos usuários.
4. Eventuais notícias ou bastidores são inseridos manualmente via CRUD administrativo para complementar o feed RSS.

O projeto está configurado para deploy imediato, utilizando as definições de container contidas no **Dockerfile** e **docker-compose.yml**.